

Unified Accelerator Fabric (UAF)

Engineering Specification for Implementation (UAF-SPEC-001)

UAF Working Group

2026-09-29 — Version 2.0 (Approved for Implementation)

Contents

Document Control	3
1 Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Design Principles	4
1.4 References	4
2 Terminology and Naming	4
2.1 Backend Codewords	4
3 System Architecture	5
3.1 Layer Diagram	5
3.2 Compile-Time Configuration (uaf_config.h)	5
4 Core Data Structures	6
4.1 Handles and Public Descriptor (uaf_types.h)	6
4.2 Enumerations	7
4.3 Work Request and Queue Pair Attributes	8
4.4 Connection and Peer Mapping Info	8
4.5 DST Structures (16-Byte Aligned CQE)	9
4.6 PAL Function Table (uaf_pal.h)	9
5 RMT Protocol Specification	10
5.1 Packet Format (Wire)	10
5.2 Field Definitions	11
5.3 Opcode Semantics	12
5.4 Connection Establishment Sequence	12
5.5 Reliability and Congestion Control	12
6 DST Protocol Specification	13
6.1 Submission Queue Entry (64 Bytes)	13
6.2 Completion Queue Entry (16 Bytes)	13
6.3 Doorbell Mechanism and State Machine	14
7 Platform Backend: UAF-N (NVIDIA)	14
7.1 Device Discovery and GPUDirect RDMA Registration	14
7.2 DST via cuFile Batch I/O	15

8 Platform Backend: UAF-S (Snapdragon)	16
8.1 Userspace Memory Registration via DMA-Heap and FastRPC SMMU	16
8.2 Software Queue Pair and FastRPC Storage Submit	17
9 Platform Backend: UAF-D (Dragonfly)	17
9.1 CXL Memory Mapping and 32-Bit Key Registration	17
9.2 Software Queue Pair over CXL.mem (Ordered Doorbell Fix)	18
9.3 DST via CXL-Mapped NVMe (Ordered SQTDBL Fix)	19
10 State Machines, Memory Management, and Error Handling	19
10.1 QP Transition Rules	19
10.2 Key Encoding (32-Bit Aligned Across All Backends)	20
11 Security Architecture and Performance Targets	20
11.1 Threat Mitigation Matrix	20
11.2 Performance Targets	20
12 Build System and Testing	20
12.1 CMakeLists.txt	20
13 Appendices	21
13.1 Appendix A: Full Public Header (uaf_core.h)	21
13.2 Appendix B: Reference Application (rdma_write_demo.c)	22

Document Control

Version	Date	Author	Changes
0.9	—	UAF WG	Initial draft
1.0	2026-01-01	UAF WG	Full engineering specification, PDF-ready
2.0	2026-09-29	UAF WG	Fixed CXL/NVMe doorbell ordering; replaced kernel DMA-BUF calls with userspace dma-heap + FastRPC SMMU mapping; completed C ABI (uaf_mr, uaf_qp_attr, uaf_peer_info); aligned 32-bit rkey wire layout and 16-byte packed DST CQE

Classification: Engineering Internal

Target Platforms: NVIDIA (H100/B200/GB200), Qualcomm Snapdragon (X Elite/8 Gen), Qualcomm Dragonfly (C1000)

1 Introduction

1.1 Purpose

This document specifies the **Unified Accelerator Fabric (UAF)**, a portable C11 framework that provides two capabilities across heterogeneous accelerator platforms:

- **Remote Memory Transport (RMT)** — direct memory access between compute nodes without CPU mediation in the data path.
- **Direct Storage Transport (DST)** — accelerator-initiated storage I/O to NVMe devices.

1.2 Scope

UAF targets three platform backends:

Backend	Silicon	Transport	Storage
UAF-N	NVIDIA H100/B200/GB200 + ConnectX-7/8	InfiniBand Verbs, RoCEv2, GPUDirect RDMA	cuFile, DOCA GPUNetIO
UAF-S	Snapdragon X Elite / 8 Gen 3-4	DMA-BUF, PCIe P2P, TCP/UDP (io_uring)	DirectStorage 1.4, FastRPC
UAF-D	Dragonfly C1000	CXL.mem, CXL.cache, PCIe Gen7	CXL-mapped NVMe

1.3 Design Principles

1. **Single C11 ABI** — Applications compile once against `uaf_core.h` and link against a platform backend (`libuaf_n.so`, `libuaf_s.so`, or `libuaf_d.so`).
2. **No CPU in data path** (where hardware permits) — UAF-N and UAF-D bypass the host CPU in the steady-state data path; UAF-S uses zero-copy SMMU-mapped rings or DSP offloads.
3. **Zero-copy by default** — All buffer paths use DMA-BUF, GPUDirect RDMA/Storage, or CXL.mem.
4. **Fail-explicit** — Every API returns a typed enum `uaf_error` code; no silent degradation.
5. **Wire-compatible** — UAF-RMT packets interop across all three backends over Ro-CEv2/UDP.

1.4 References

- InfiniBand Architecture Specification Vol. 1, Release 1.4
- CXL Specification 3.1 (CXL Consortium)
- NVM Express Base Specification, Revision 2.0d
- NVIDIA DOCA GPUNetIO Programming Guide v2.5
- Microsoft DirectStorage 1.4 API Reference
- Qualcomm FastRPC User Guide (80-NH713-1)
- Linux `rdma-core` v50, `libibverbs` API, and `<linux/dma-heap.h>`

2 Terminology and Naming

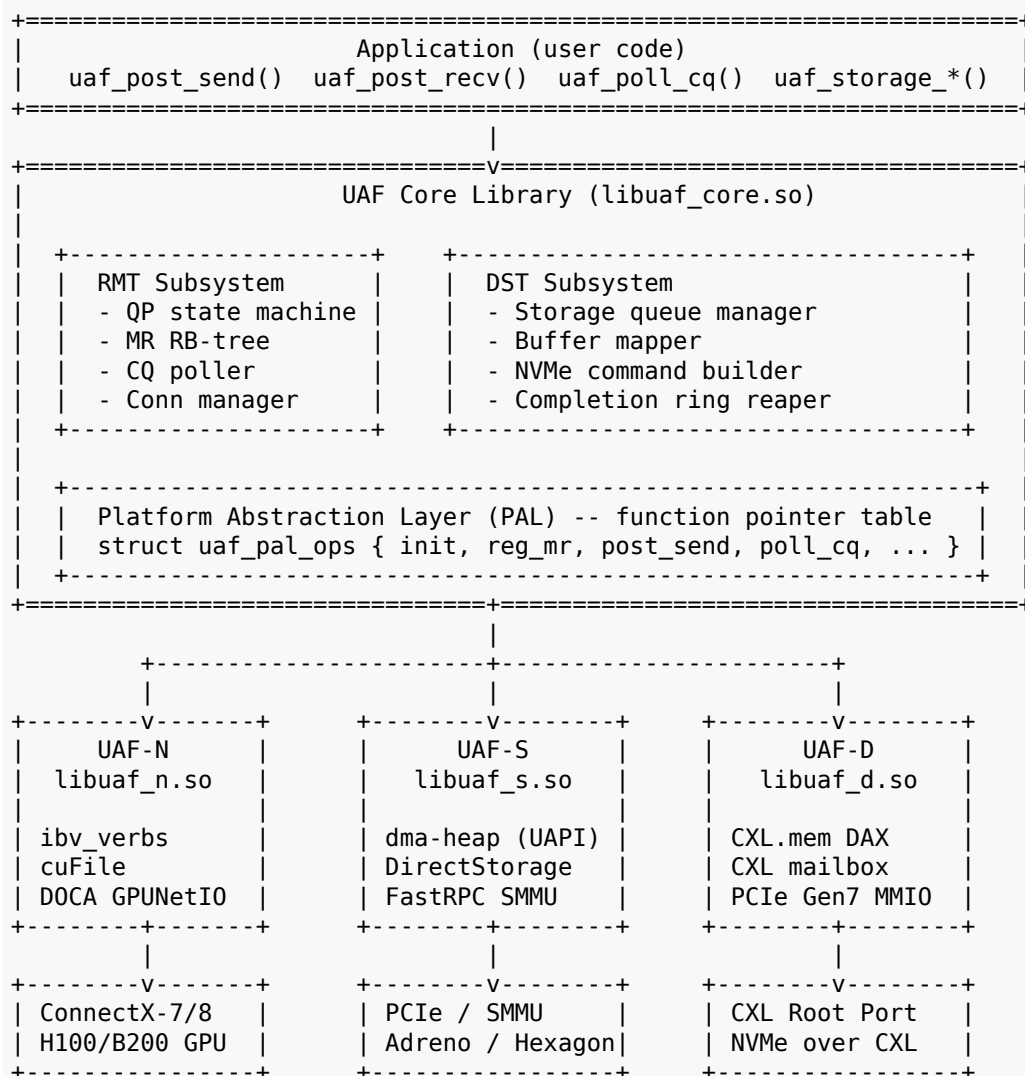
Term	Definition
RMT	Remote Memory Transport — RDMA-equivalent
DST	Direct Storage Transport — GPUDirect Storage / DirectStorage equivalent
UAF	Unified Accelerator Fabric
PAL	Platform Abstraction Layer
QP	Queue Pair (send + receive queues)
CQ	Completion Queue
MR	Memory Region
SQ	Storage Queue (DST submission)
CR	Completion Ring (DST completion)
SGE	Scatter-Gather Element
WR	Work Request
WC	Work Completion

2.1 Backend Codewords

Codeword	Platform
UAF-N	NVIDIA (ConnectX + Hopper/Blackwell)
UAF-S	Qualcomm Snapdragon (Adreno + Hexagon DSP)
UAF-D	Qualcomm Dragonfly (C1000 CXL Accelerator)

3 System Architecture

3.1 Layer Diagram



3.2 Compile-Time Configuration (uaf_config.h)

```

#ifndef UAF_CONFIG_H
#define UAF_CONFIG_H

#define UAF_VERSION_MAJOR    2
#define UAF_VERSION_MINOR    0
#define UAF_VERSION_PATCH    0

/* Feature flags */
#define UAF_ENABLE_ATOMICS    1
#define UAF_ENABLE_DST        1
#define UAF_ENABLE_CXL_CACHE  0
#define UAF_ENABLE_DEBUG      0

```

```

/* Resource limits */
#define UAF_MAX_QP            4096
#define UAF_MAX_CQ            4096
#define UAF_MAX_MR            65536
#define UAF_MAX_SGE           16
#define UAF_MAX_INLINE        256
#define UAF_CQ_DEPTH_DEFAULT  1024
#define UAF_SQ_DEPTH_DEFAULT   256

/* Wire protocol constants */
#define UAF_WIRE_MAGIC         0x55414652U /* "UAFR" */
#define UAF_WIRE_VERSION       2
#define UAF_WIRE_HDR_SIZE      64
#define UAF_WIRE_MTU_DEFAULT   4096

/* Wire header flags */
#define UAF_WIRE_FLAG_INLINE   (1U << 0)
#define UAF_WIRE_FLAG_IMM      (1U << 1)
#define UAF_WIRE_FLAG_SOLICITED (1U << 2)
#define UAF_WIRE_FLAG_ACK_REQ  (1U << 3)

#endif /* UAF_CONFIG_H */

```

4 Core Data Structures

4.1 Handles and Public Descriptor (uaf_types.h)

```

#ifndef UAF_TYPES_H
#define UAF_TYPES_H

#include <stdint.h>
#include <stddef.h>

typedef struct uaf_device    uaf_device_t;
typedef struct uaf_domain    uaf_domain_t;
typedef struct uaf_qp        uaf_qp_t;
typedef struct uaf_cq        uaf_cq_t;
typedef struct uaf_sq        uaf_sq_t;
typedef struct uaf_cr        uaf_cr_t;

/* Public Memory Region descriptor (v2.0) */
typedef struct uaf_mr {
    uaf_domain_t *pd;
    void *addr;
    size_t length;
    uint32_t lkey;
    uint32_t rkey;
    uint64_t flags;
    void *pal_priv; /* Backend-private state */
} uaf_mr_t;

```

4.2 Enumerations

```
enum uaf_qp_state {
    UAF_QPS_RESET = 0,
    UAF_QPS_INIT = 1,
    UAF_QPS_RTR = 2,
    UAF_QPS_RTS = 3,
    UAF_QPS_SQD = 4,
    UAF_QPS_SQE = 5,
    UAF_QPS_ERR = 6,
};

enum uaf_wr_opcode {
    UAF_WR_SEND = 0x01,
    UAF_WR_SEND_INLINE = 0x02,
    UAF_WR_RECV = 0x03,
    UAF_WR_RDMA_READ = 0x04,
    UAF_WR_RDMA_WRITE = 0x05,
    UAF_WR_RDMA_WRITE_IMM = 0x06,
    UAF_WR_ATOMIC_CMP_SWP = 0x07,
    UAF_WR_ATOMIC_FETCH_ADD = 0x08,
    UAF_WR_STORAGE_READ = 0x10,
    UAF_WR_STORAGE_WRITE = 0x11,
    UAF_WR_STORAGE_FLUSH = 0x12,
};

enum uaf_mr_flags {
    UAF_MR_LOCAL_WRITE = (1ULL << 0),
    UAF_MR_REMOTE_WRITE = (1ULL << 1),
    UAF_MR_REMOTE_READ = (1ULL << 2),
    UAF_MR_ATOMIC = (1ULL << 3),
    UAF_MR_GPU = (1ULL << 4),
    UAF_MR_CXL = (1ULL << 5),
};

enum uaf_send_flags {
    UAF_SEND_SIGNALED = (1U << 0),
    UAF_SEND_INLINE = (1U << 1),
    UAF_SEND_SOLICITED = (1U << 2),
    UAF_SEND_FENCE = (1U << 3),
};

enum uaf_error {
    UAF_OK = 0,
    UAF_ERR_INVALID = -1,
    UAF_ERR_NOMEM = -2,
    UAF_ERR_NODEV = -3,
    UAF_ERR_PERM = -4,
    UAF_ERR_BUSY = -5,
    UAF_ERR_TIMEOUT = -6,
    UAF_ERR_QP_STATE = -7,
    UAF_ERR_CQ_EMPTY = -8,
    UAF_ERR_MR_FAULT = -9,
    UAF_ERR_NOT_SUPPORTED = -10,
    UAF_ERR_PROTO = -11,
    UAF_ERR_CRC = -12,
    UAF_ERR_REMOTE = -13,
};
```

4.3 Work Request and Queue Pair Attributes

```
struct uaf_sge {
    uint64_t addr;        /* Virtual address */
    uint32_t length;      /* Length in bytes */
    uint32_t lkey;        /* Local memory key */
};

struct uaf_wr {
    uint64_t wr_id;        /* Opaque user ID */
    enum uaf_wr_opcode opcode;
    int num_sge;
    struct uaf_sge *sge;
    uint32_t send_flags; /* enum uaf_send_flags bitmask */
    uint32_t imm_data;   /* For WRITE_IMM / SEND */
    uint64_t remote_addr; /* Target VA or CXL offset */
    uint32_t rkey;        /* 32-bit remote protection key */
    uint32_t reserved;
    /* Atomics */
    uint64_t compare_add;
    uint64_t swap;
};

struct uaf_wc {
    uint64_t wr_id;
    enum uaf_wr_opcode opcode;
    enum uaf_error status;
    uint32_t byte_len;
    uint32_t qp_num;
    uint32_t src_qp;
    uint32_t imm_data;
};

struct uaf_qp_attr {
    enum uaf_qp_state qp_state;
    uint32_t sq_psn;
    uint32_t rq_psn;
    uint32_t dest_qp_num;
    uint16_t dest_lid;
    uint8_t dest_gid[16];
    uint16_t path_mtu;
    uint8_t max_rd_atomic;
    uint8_t retry_cnt;
    uint8_t rnr_retry;
    uint8_t port_num;
};
```

4.4 Connection and Peer Mapping Info

```
struct uaf_conn_info {
    uint32_t qp_num;
    uint32_t psn;        /* Initial Packet Sequence Number */
    uint16_t lid;        /* InfiniBand Local ID */
    uint8_t gid[16];     /* Global ID (RoCEv2 / IPv6) */
    uint32_t rkey;        /* Remote key for peer MR */
    uint64_t remote_va;   /* Remote VA or CXL offset */
    uint16_t mtu;
```

```

uint8_t    sl;                /* Service level */
uint8_t    traffic_class;
/* UAF-D extension */
uint64_t    cxl_base;
uint64_t    cxl_size;
/* UAF-S extension */
int32_t     dma_buf_fd;
uint32_t     reserved;
};

struct uaf_peer_info {
    uint64_t    remote_va;
    uint64_t    length;
    uint32_t    rkey;
    int32_t     dma_buf_fd;    /* UAF-S local/IPC FD; -1 if unused */
    uint64_t    cxl_base;     /* UAF-D CXL window offset */
};

```

4.5 DST Structures (16-Byte Aligned CQE)

```

struct uaf_storage_cmd {
    uint64_t    wr_id;
    uint64_t    lba;          /* Starting Logical Block Address */
    uint32_t    nlb;          /* Number of logical blocks (1-based) */
    uint32_t    buf_offset;
    void        *buf;         /* DMA-BUF, CXL, or cuFile buffer */
    size_t      buf_len;
    enum uaf_wr_opcode opcode; /* STORAGE_READ / WRITE / FLUSH */
    uint32_t    flags;
};

/* Matches the 16-byte wire/ring CQE layout of Section 6.2 exactly */
struct __attribute__((packed)) uaf_storage_cqe {
    uint16_t    cmd_id;        /* Lower 16 bits of wr_id */
    int16_t     status;        /* enum uaf_error cast to int16_t */
    uint32_t    bytes_transferred;
    uint32_t    latency_ns;    /* saturates at ~4.29 s */
    uint32_t    reserved;
};

_Static_assert(sizeof(struct uaf_storage_cqe) == 16,
    "uaf_storage_cqe must be exactly 16 bytes");

#endif /* UAF_TYPES_H */

```

4.6 PAL Function Table (uaf_pal.h)

```

#ifndef UAF_PAL_H
#define UAF_PAL_H

#include "uaf_types.h"

struct uaf_pal_ops {
    const char *name;

```

```

int (*init)(void);
int (*fini)(void);

int (*get_device_list)(uaf_device_t ***devs, int *count);
int (*open_device)(uaf_device_t *dev, uaf_domain_t **pd);
int (*close_device)(uaf_device_t *dev);

int (*reg_mr)(uaf_domain_t *pd, void *addr, size_t len,
              uint64_t flags, uaf_mr_t **mr);
int (*dereg_mr)(uaf_mr_t *mr);

int (*create_cq)(uaf_domain_t *pd, int cqe, uaf_cq_t **cq);
int (*destroy_cq)(uaf_cq_t *cq);

int (*create_qp)(uaf_domain_t *pd, uaf_cq_t *scq,
                 uaf_cq_t *rcq, uaf_qp_t **qp);
int (*modify_qp)(uaf_qp_t *qp, enum uaf_qp_state st,
                 struct uaf_qp_attr *attr);
int (*destroy_qp)(uaf_qp_t *qp);

int (*post_send)(uaf_qp_t *qp, struct uaf_wr *wr);
int (*post_recv)(uaf_qp_t *qp, struct uaf_wr *wr);
int (*poll_cq)(uaf_cq_t *cq, int max, struct uaf_wc *wc);

int (*storage_open)(uaf_domain_t *pd, const char *path,
                    uaf_sq_t **sq, uaf_cr_t **cr);
int (*storage_submit)(uaf_sq_t *sq, struct uaf_storage_cmd *cmd);
int (*storage_poll)(uaf_cr_t *cr, int max,
                    struct uaf_storage_cqe *cqe);
int (*storage_close)(uaf_sq_t *sq);

int (*map_peer)(uaf_domain_t *pd, const struct uaf_peer_info *peer,
                uaf_mr_t **mr);
int (*unmap_peer)(uaf_mr_t *mr);
};

extern const struct uaf_pal_ops uaf_pal_nvidia;
extern const struct uaf_pal_ops uaf_pal_snapdragon;
extern const struct uaf_pal_ops uaf_pal_dragonfly;

#endif /* UAF_PAL_H */

```

5 RMT Protocol Specification

5.1 Packet Format (Wire)

All multi-byte fields are encoded in network byte order (big-endian). The fixed header is 64 bytes.

0	1	2	3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1			
+--+			
			0..3
+--+			
version (2)			opcode
			flags
			4..7

qp_id	8..11
msg_seq	12..15
lkey_rsvd	16..19
rkey	20..23
remote_va (8 bytes)	24..31
length	32..35
imm_data	36..39
crc32c	40..43
reserved (20 bytes)	44..63
payload (0..MTU bytes, or 0..256B inline)	64+

5.2 Field Definitions

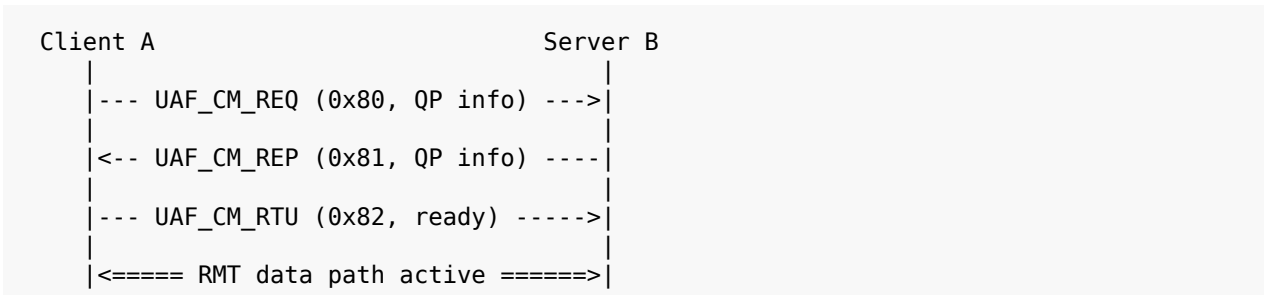
Field	Offset	Size (B)	Description
magic	0	4	0x55414652 ("UAFR")
version	4	1	Protocol version (2)
opcode	5	1	enum uaf_wr_opcode or CM control opcode
flags	6	2	Bit 0: inline; Bit 1: immediate; Bit 2: solicited; Bit 3: ack_req
qp_id	8	4	Destination Queue Pair number (uint32_t)
msg_seq	12	4	Monotonic Packet Sequence Number (PSN)
lkey_rsvd	16	4	Reserved on wire (set to 0 by sender, ignored by receiver)
rkey	20	4	32-bit Remote Memory Key (uint32_t, matches uaf_wr.rkey)
remote_va	24	8	Target virtual address or CXL window offset
length	32	4	Payload length in bytes following byte 63 (including inline data)
imm_data	36	4	Optional 32-bit immediate data
crc32c	40	4	Castagnoli CRC32C over header bytes 0..39

Field	Offset	Size (B)	Description
reserved	44	20	Zero-padded to align header to 64-byte cacheline

5.3 Opcode Semantics

Opcode	Direction	Requires rkey	Payload Following Header
UAF_WR_SEND	Local -> Remote	No	0..MTU bytes from SGE
UAF_WR_SEND_INLINE	Local -> Remote	No	1..256 bytes (UAF_MAX_INLINE) immediately after byte 63
UAF_WR_RECV	Remote -> Local	No	None (local receive buffer descriptor only)
UAF_WR_RDMA_READ	Remote -> Local	Yes	Request: 0 B; Response: length bytes
UAF_WR_RDMA_WRITE	Local -> Remote	Yes	length bytes written to remote_va
UAF_WR_RDMA_WRITE_IMM	Local -> Remote	Yes	length bytes + imm_data delivered to remote CQ
UAF_WR_ATOMIC_CMP_SWP	Local -> Remote	Yes	16 bytes (compare_add + swap, 8 B each)
UAF_WR_ATOMIC_FETCH_ADD	Local -> Remote	Yes	8 bytes (compare_add)

5.4 Connection Establishment Sequence



5.5 Reliability and Congestion Control

- PSN (Packet Sequence Number) is carried in msg_seq with Go-Back-N retransmission (window size 64).
- Control packets: opcode = 0x90 (ACK), 0x91 (NAK with expected PSN in imm_data).
- Retransmission timer: 100 ms base timeout, 8 retries max, transitioning the QP to UAF_QPS_ERR with UAF_ERR_TIMEOUT.

Congestion control:

- **UAF-N:** Offloaded to ConnectX hardware (DCQCN / RoCEv2 congestion management).
- **UAF-S / UAF-D:** Software DCQCN rate limiter:

$$R_{\text{new}} = R_{\text{cur}} \times \left(1 - \frac{\alpha}{2}\right) \quad (\text{on ECN mark})$$

$$R_{\text{new}} = R_{\text{cur}} + R_{\text{AI}} \quad (\text{on clean ACK})$$

6 DST Protocol Specification

6.1 Submission Queue Entry (64 Bytes)

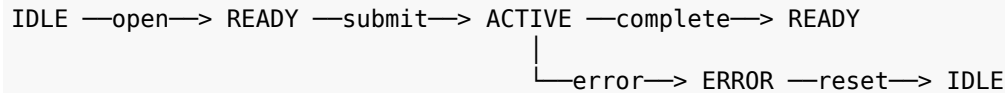
0	1	2	3
0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9	0 1
+	+	+	+
opcode	flags	cmd_id (2 bytes)	0..3
+	+	+	+
	nsid (4 bytes)		4..7
+	+	+	+
	lba (8 bytes)		8..15
+	+	+	+
	nlb (2 bytes)	rsvd (2 bytes)	16..19
+	+	+	+
	data_ptr (8 bytes, physical/IOVA)		20..27
+	+	+	+
	data_len (4 bytes)		28..31
+	+	+	+
	meta_ptr (8 bytes)		32..39
+	+	+	+
	meta_len (4 bytes)	rsvd2 (4 bytes)	40..47
+	+	+	+
	crc32c (4 bytes)		48..51
+	+	+	+
	reserved (12 bytes)		52..63
+	+	+	+

6.2 Completion Queue Entry (16 Bytes)

0	1	2	3
0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9	0 1
+	+	+	+
cmd_id (2 bytes)	status (2 bytes)		0..3
+	+	+	+
	bytes_transferred (4 bytes)		4..7
+	+	+	+
	latency_ns (4 bytes)		8..11
+	+	+	+
	reserved (4 bytes)		12..15
+	+	+	+

6.3 Doorbell Mechanism and State Machine

Backend	Doorbell Mechanism
UAF-N	cuFileBatchIOSubmit (batches NVMe SQ tail updates via nvidia-fs / BAR0)
UAF-S	DirectStorage IDStorageQueue::Submit() (Windows) or FastRPC DSP invoke (Linux)
UAF-D	Release-ordered store to sq_tail followed by MMIO write of the new tail to cxl_bar + NVME_SQ0TDBL



7 Platform Backend: UAF-N (NVIDIA)

7.1 Device Discovery and GPUDirect RDMA Registration

```

static int nv_get_device_list(uaf_device_t ***devs, int *count)
{
    struct ibv_device **list = ibv_get_device_list(count);
    if (!list || *count <= 0) return UAF_ERR_NODEV;

    *devs = calloc((size_t)*count, sizeof(uaf_device_t *));
    if (!*devs) {
        ibv_free_device_list(list);
        return UAF_ERR_NOMEM;
    }

    for (int i = 0; i < *count; i++) {
        uaf_device_t *d = calloc(1, sizeof(*d));
        d->ibv_dev = list[i];
        d->name = strdup(ibv_get_device_name(list[i]));
        (*devs)[i] = d;
    }
    ibv_free_device_list(list);
    return UAF_OK;
}

static int nv_reg_mr(uaf_domain_t *pd, void *addr, size_t len,
                    uint64_t flags, uaf_mr_t **out)
{
    int access = 0;
    if (flags & UAF_MR_LOCAL_WRITE) access |= IBV_ACCESS_LOCAL_WRITE;
    if (flags & UAF_MR_REMOTE_WRITE) access |= IBV_ACCESS_REMOTE_WRITE;
    if (flags & UAF_MR_REMOTE_READ) access |= IBV_ACCESS_REMOTE_READ;
    if (flags & UAF_MR_ATOMIC) access |= IBV_ACCESS_REMOTE_ATOMIC;

    struct ibv_mr *ib_mr = ibv_reg_mr(pd->ibv_pd, addr, len, access);
    if (!ib_mr) return UAF_ERR_NOMEM;
}

```

```

uaf_mr_t *m = calloc(1, sizeof(*m));
if (!m) {
    ibv_dereg_mr(ib_mr);
    return UAF_ERR_NOMEM;
}
m->pd      = pd;
m->addr     = addr;
m->length   = len;
m->lkey     = ib_mr->lkey;
m->rkey     = ib_mr->rkey;
m->flags    = flags;
m->pal_priv = ib_mr;
*out = m;
return UAF_OK;
}

```

7.2 DST via cuFile Batch I/O

```

static int nv_storage_open(uaf_domain_t *pd, const char *path,
                           uaf_sq_t **out_sq, uaf_cr_t **out_cr)
{
    CUfileDescr_t desc = {0};
    desc.type = CU_FILE_HANDLE_TYPE_OPAQUE_FD;
    desc.handle.fd = open(path, O_RDWR | O_DIRECT | O_CLOEXEC);
    if (desc.handle.fd < 0) return UAF_ERR_NODEV;

    CUfileHandle_t fh;
    CUfileError_t err = cuFileHandleRegister(&fh, &desc);
    if (err.err != CU_FILE_SUCCESS) {
        close(desc.handle.fd);
        return UAF_ERR_NODEV;
    }

    CUfileBatchHandle_t batch;
    err = cuFileBatchIOSetUp(&batch, UAF_SQ_DEPTH_DEFAULT);
    if (err.err != CU_FILE_SUCCESS) {
        cuFileHandleDeregister(fh);
        close(desc.handle.fd);
        return UAF_ERR_NOMEM;
    }

    uaf_sq_t *sq = calloc(1, sizeof(*sq));
    uaf_cr_t *cr = calloc(1, sizeof(*cr));
    sq->cu_file_handle = fh;
    sq->batch_handle   = batch;
    sq->fd              = desc.handle.fd;
    cr->batch_handle    = batch;

    *out_sq = sq;
    *out_cr = cr;
    return UAF_OK;
}

```

8 Platform Backend: UAF-S (Snapdragon)

8.1 Userspace Memory Registration via DMA-Heap and FastRPC SMMU

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <linux/dma-heap.h>
#include "remote.h"

#define FASTRPC_ATTR_NON_COHERENT 1

struct uaf_s_mr_priv {
    int dma_buf_fd;
    int owns_mmap;
};

static int sd_reg_mr(uaf_domain_t *pd, void *addr, size_t len,
                    uint64_t flags, uaf_mr_t **out)
{
    int heap_fd = open("/dev/dma_heap/system", O_RDWR | O_CLOEXEC);
    if (heap_fd < 0) return UAF_ERR_NODEV;

    struct dma_heap_allocation_data alloc = {
        .len = len,
        .fd_flags = O_RDWR | O_CLOEXEC,
    };
    if (ioctl(heap_fd, DMA_HEAP_IOCTL_ALLOC, &alloc) < 0) {
        close(heap_fd);
        return UAF_ERR_NOMEM;
    }
    close(heap_fd);

    void *dmabuf_va = mmap(NULL, len, PROT_READ | PROT_WRITE,
                           MAP_SHARED, alloc.fd, 0);
    if (dmabuf_va == MAP_FAILED) {
        close(alloc.fd);
        return UAF_ERR_NOMEM;
    }

    if (addr && addr != dmabuf_va) {
        memcpy(dmabuf_va, addr, len);
    }

    /* Map DMA-BUF FD into the Hexagon DSP SMMU context table */
    remote_register_buf_attr(dmabuf_va, (int)len, alloc.fd,
                            FASTRPC_ATTR_NON_COHERENT);

    struct uaf_s_mr_priv *priv = calloc(1, sizeof(*priv));
    uaf_mr_t *mr = calloc(1, sizeof(*mr));
    priv->dma_buf_fd = alloc.fd;
    priv->owns_mmap = 1;

    mr->pd = pd;
    mr->addr = dmabuf_va;
}
```

```

mr->length    = len;
mr->lkey       = (uint32_t)alloc.fd;
mr->rkey       = (uint32_t)alloc.fd ^ 0xA5A5A5AU;
mr->flags      = flags;
mr->pal_priv   = priv;
*out = mr;
return UAF_OK;
}

```

8.2 Software Queue Pair and FastRPC Storage Submit

```

#include <stdatomic.h>

struct uaf_s_ring {
    _Atomic uint32_t head;
    _Atomic uint32_t tail;
    uint32_t size; /* Power of two */
    uint32_t pad[5]; /* Align to 32-byte boundary */
    struct uaf_wr entries[];
};

static int sd_post_send(uaf_qp_t *qp, struct uaf_wr *wr)
{
    struct uaf_s_ring *sq = qp->send_ring;
    uint32_t head = atomic_load_explicit(&sq->head, memory_order_relaxed);
    uint32_t tail = atomic_load_explicit(&sq->tail, memory_order_acquire);
    uint32_t next = (head + 1U) & (sq->size - 1U);

    if (next == tail) return UAF_ERR_BUSY;

    sq->entries[head] = *wr;
    atomic_store_explicit(&sq->head, next, memory_order_release);

    if (qp->doorbell) {
        atomic_thread_fence(memory_order_seq_cst);
        *(volatile uint32_t *)qp->doorbell = qp->qp_num;
    } else {
        return sd_send_via_socket(qp, wr);
    }
    return UAF_OK;
}

```

9 Platform Backend: UAF-D (Dragonfly)

9.1 CXL Memory Mapping and 32-Bit Key Registration

```

struct uaf_d_mr_priv {
    int dax_fd;
    uint64_t cxl_base;
};

static int df_reg_mr(uaf_domain_t *pd, void *addr, size_t len,

```

```

        uint64_t flags, uaf_mr_t **out)
{
    int fd = open("/dev/dax0.0", O_RDWR | O_CLOEXEC);
    if (fd < 0) return UAF_ERR_NODEV;

    uint64_t cxl_offset = df_alloc_cxl_window(pd, len);
    void *cxl_va = mmap(addr, len, PROT_READ | PROT_WRITE,
                        MAP_SHARED | MAP_SYNC, fd, (off_t)cxl_offset);
    if (cxl_va == MAP_FAILED) {
        close(fd);
        return UAF_ERR_NOMEM;
    }

    uint32_t key = df_generate_rkey(pd, cxl_offset, len, flags);

    struct uaf_d_mr_priv *priv = calloc(1, sizeof(*priv));
    uaf_mr_t *mr = calloc(1, sizeof(*mr));
    priv->dax_fd = fd;
    priv->cxl_base = cxl_offset;

    mr->pd = pd;
    mr->addr = cxl_va;
    mr->length = len;
    mr->lkey = key;
    mr->rkey = key;
    mr->flags = flags;
    mr->pal_priv = priv;
    *out = mr;
    return UAF_OK;
}

```

9.2 Software Queue Pair over CXL.mem (Ordered Doorbell Fix)

```

#include <stdatomic.h>

static inline void df_ring_doorbell(uaf_qp_t *qp, uint32_t which)
{
    atomic_thread_fence(memory_order_seq_cst);
    volatile uint64_t *mailbox =
        (volatile uint64_t *)((uintptr_t)qp->cxl_mailbox_base + 0x100ULL);
    *mailbox = ((uint64_t)qp->qp_num << 32) | (uint64_t)which;
}

static int df_post_send(uaf_qp_t *qp, struct uaf_wr *wr)
{
    struct uaf_d_qp *d = qp->cxl_qp;
    uint32_t cur = atomic_load_explicit(&d->sq_head, memory_order_relaxed);
    uint32_t tail = atomic_load_explicit(&d->sq_tail, memory_order_acquire);
    uint32_t next = (cur + 1U) % UAF_CQ_DEPTH_DEFAULT;

    if (next == tail) return UAF_ERR_BUSY;

    d->sq[cur] = *wr;
    atomic_store_explicit(&d->sq_head, next, memory_order_release);
    df_ring_doorbell(qp, UAF_DOORBELL_SQ);
    return UAF_OK;
}

```

```
}
```

9.3 DST via CXL-Mapped NVMe (Ordered SQTDBL Fix)

```
static int df_storage_submit(uaf_sq_t *sq, struct uaf_storage_cmd *cmd)
{
    uint32_t tail = sq->sq_tail;
    uint32_t next_tail = (tail + 1U) % sq->depth;

    if (next_tail ==
        atomic_load_explicit(&sq->sq_head, memory_order_acquire)) {
        return UAF_ERR_BUSY;
    }

    struct nvme_sq_entry *e = &sq->nvme_sq[tail];
    memset(e, 0, sizeof(*e));
    e->opcode = (cmd->opcode == UAF_WR_STORAGE_READ) ?
        NVME_OPC_READ : NVME_OPC_WRITE;
    e->nsid = sq->nsid;
    e->dptr_prpl = (uint64_t)((uintptr_t)cmd->buf + cmd->buf_offset);
    e->cdwl0 = (uint32_t)(cmd->lba & 0xFFFFFFFFFULL);
    e->cdwl1 = (uint32_t)(cmd->lba >> 32);
    e->cdwl2 = (uint32_t)((cmd->nlb - 1U) & 0xFFFFU);
    e->cmd_id = (uint16_t)(cmd->wr_id & 0xFFFFU);

    sq->sq_tail = next_tail;
    atomic_thread_fence(memory_order_seq_cst);

    volatile uint32_t *db =
        (volatile uint32_t *)((uintptr_t)sq->cxl_bar + sq->sq_tdbl_offset);
    *db = next_tail;

    return UAF_OK;
}
```

10 State Machines, Memory Management, and Error Handling

10.1 QP Transition Rules

From	To	Trigger	Validation
RESET	INIT	modify_qp(INIT)	PD and CQs attached
INIT	RTR	modify_qp(RTR)	Remote QP number, MTU, and GID/LID valid
RTR	RTS	modify_qp(RTS)	sq_psn and retry counters configured
RTS	SQD	modify_qp(SQD)	Drain in-flight send WRs
SQD	RTS	modify_qp(RTS)	Send queue drained
Any	ERR	Transport/MR fault	Flush all queued WRs with UAF_ERR_QP_STATE
ERR	RESET	modify_qp(RESET)	Ring indices reset to 0

10.2 Key Encoding (32-Bit Aligned Across All Backends)

Backend	mr->lkey (32-bit)	mr->rkey (32-bit)	Address Carrier (remote_va)
UAF-N	ibv_mr->lkey	ibv_mr->rkey	64-bit GPU/host virtual address
UAF-S	dma_buf_fd	Salted token (fd ^ salt)	64-bit SMMU/ring offset
UAF-D	CXL MR table token	32-bit random protection key	64-bit cxl_base window offset

11 Security Architecture and Performance Targets

11.1 Threat Mitigation Matrix

Threat	Mitigation
Unauthorized remote RDMA write	32-bit CSPRNG rkey validation + bounds check + IOMMU/SMMU
Packet injection / corruption	Header CRC32C + optional PSP/IPsec/MACsec hardware offload
Malicious DMA from peripheral	Strict per-PD IOMMU / ARM SMMU stage-1 translation tables
Physical CXL bus snooping	CXL 3.1 Integrity and Data Encryption (IDE) on UAF-D links

11.2 Performance Targets

Metric	UAF-N (IB / RoCEv2)	UAF-S (PCIe / UDP)	UAF-D (CXL 3.1)
4 KB RMT one-way latency	1.5 us / 2.5 us	8.0 us / 25.0 us	0.3 us
Peak unidirectional bandwidth	400 Gbps / 800 Gbps	64 Gbps	2,000 Gbps (2 Tbps)
DST random read IOPS (4 KB)	200 M+ IOPS	1.2 M IOPS	50 M IOPS

12 Build System and Testing

12.1 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(uaf VERSION 2.0.0 LANGUAGES C CXX)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_POSITION_INDEPENDENT_CODE ON)
```

```

option(UAF_BUILD_NVIDIA      "Build NVIDIA backend (UAF-N)"      OFF)
option(UAF_BUILD_SNAPDRAGON  "Build Snapdragon backend (UAF-S)"  OFF)
option(UAF_BUILD_DRAGONFLY   "Build Dragonfly backend (UAF-D)"   OFF)
option(UAF_BUILD_TESTS       "Build unit and interop tests"      ON)

add_library(uaf_core SHARED
    src/core/uaf_init.c src/core/uaf_qp.c src/core/uaf_cq.c
    src/core/uaf_mr.c   src/core/uaf_conn.c src/core/uaf_storage.c
)
target_include_directories(uaf_core PUBLIC include)

```

13 Appendices

13.1 Appendix A: Full Public Header (uaf_core.h)

```

#ifndef UAF_CORE_H
#define UAF_CORE_H

#include <stdint.h>
#include <stddef.h>
#include "uaf_config.h"
#include "uaf_types.h"

#ifdef __cplusplus
extern "C" {
#endif

int uaf_init(void);
int uaf_fini(void);
const char *uaf_version(void);
const char *uaf_backend_name(void);

int uaf_get_device_list(uaf_device_t ***devs, int *count);
int uaf_free_device_list(uaf_device_t **devs, int count);
int uaf_open_device(uaf_device_t *dev, uaf_domain_t **pd);
int uaf_close_device(uaf_device_t *dev);

int uaf_reg_mr(uaf_domain_t *pd, void *addr, size_t len,
               uint64_t flags, uaf_mr_t **mr);
int uaf_dereg_mr(uaf_mr_t *mr);

int uaf_create_cq(uaf_domain_t *pd, int cqe, uaf_cq_t **cq);
int uaf_destroy_cq(uaf_cq_t *cq);
int uaf_poll_cq(uaf_cq_t *cq, int max, struct uaf_wc *wc);

int uaf_create_qp(uaf_domain_t *pd, uaf_cq_t *scq, uaf_cq_t *rcq,
                  uaf_qp_t **qp);
int uaf_modify_qp(uaf_qp_t *qp, enum uaf_qp_state state,
                  struct uaf_qp_attr *attr);
int uaf_destroy_qp(uaf_qp_t *qp);
int uaf_post_send(uaf_qp_t *qp, struct uaf_wr *wr);
int uaf_post_recv(uaf_qp_t *qp, struct uaf_wr *wr);

```

```

int uaf_get_conn_info(uaf_qp_t *qp, struct uaf_conn_info *local);
int uaf_connect(uaf_qp_t *qp, const struct uaf_conn_info *remote);
int uaf_disconnect(uaf_qp_t *qp);

int uaf_map_peer(uaf_domain_t *pd, const struct uaf_peer_info *peer,
                uaf_mr_t **mr);
int uaf_unmap_peer(uaf_mr_t *mr);

int uaf_storage_open(uaf_domain_t *pd, const char *path,
                    uaf_sq_t **sq, uaf_cr_t **cr);
int uaf_storage_submit(uaf_sq_t *sq, struct uaf_storage_cmd *cmd);
int uaf_storage_poll(uaf_cr_t *cr, int max,
                    struct uaf_storage_cqe *cqe);
int uaf_storage_close(uaf_sq_t *sq);

const char *uaf_strerror(enum uaf_error e);
void uaf_set_log_level(int level);
void uaf_set_log_callback(void (*cb)(int level, const char *msg));

#ifdef __cplusplus
}
#endif

#endif /* UAF_CORE_H */

```

13.2 Appendix B: Reference Application (rdma_write_demo.c)

```

#include "uaf_core.h"
#include <stdio.h>
#include <string.h>

int main(void)
{
    if (uaf_init() != UAF_OK) return 1;

    uaf_device_t **devs = NULL;
    int n = 0;
    if (uaf_get_device_list(&devs, &n) != UAF_OK || n == 0) return 1;

    uaf_domain_t *pd = NULL;
    uaf_open_device(devs[0], &pd);

    uaf_cq_t *cq = NULL;
    uaf_create_cq(pd, 64, &cq);

    uaf_qp_t *qp = NULL;
    uaf_create_qp(pd, cq, cq, &qp);

    char buf[4096] = "Hello, UAF v2.0!";
    uaf_mr_t *mr = NULL;
    uaf_reg_mr(pd, buf, sizeof(buf),
              UAF_MR_LOCAL_WRITE | UAF_MR_REMOTE_WRITE |
              UAF_MR_REMOTE_READ,
              &mr);

    struct uaf_qp_attr attr = {

```

```

        .qp_state      = UAF_QPS_INIT,
        .path_mtu      = UAF_WIRE_MTU_DEFAULT,
        .max_rd_atomic = 4,
        .retry_cnt     = 7,
    };
    uaf_modify_qp(qp, UAF_QPS_INIT, &attr);
    attr.qp_state = UAF_QPS_RTR;
    uaf_modify_qp(qp, UAF_QPS_RTR, &attr);
    attr.qp_state = UAF_QPS_RTS;
    uaf_modify_qp(qp, UAF_QPS_RTS, &attr);

    struct uaf_conn_info remote = {0};
    struct uaf_sge sge = {
        .addr = (uint64_t)(uintptr_t)mr->addr,
        .length = (uint32_t)mr->length,
        .lkey = mr->lkey,
    };
    struct uaf_wr wr = {
        .wr_id = 1,
        .opcode = UAF_WR_RDMA_WRITE,
        .num_sge = 1,
        .sge = &sge,
        .send_flags = UAF_SEND_SIGNALED,
        .remote_addr = remote.remote_va,
        .rkey = remote.rkey,
    };
    uaf_post_send(qp, &wr);

    struct uaf_wc wc;
    while (uaf_poll_cq(cq, 1, &wc) == UAF_ERR_CQ_EMPTY)
        ;

    printf("Write completed: status=%s len=%u\n",
        uaf_strerror(wc.status), wc.byte_len);

    uaf_dereg_mr(mr);
    uaf_destroy_qp(qp);
    uaf_destroy_cq(cq);
    uaf_close_device(devs[0]);
    uaf_free_device_list(devs, n);
    uaf_fini();
    return 0;
}

```